8.11.2018

CSE 3017 DIGITAL DESIGN
MIDTERM SOLUTION MANUAL

Name:

Q1) (25 points) Truth table of a logic function w=f(x,y,z) is given below. Implement it by

x	y	z	w
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

a-) (10 points) Dataflow modeling in Verilog
b-) (10 points) Behavioral modeling (using conditional statements) in Verilog
c-) (5 points) Three select pin multiplexer in hardware form.

SOLUTION:

```
module first_question(w,x,y,z);

input x,y,z;

//for part a
//output w;
//assign w=(~x&~y&z)|(~x&y&z)|(x&~y&z)|(x&y&z);

//for part b
output reg w;

always @ (x,y,z)
  case ({x,y,z})
    3'b001, 3'b011, 3'b101, 3'b111 : w=1'b1;
    default: w=1'b0;
   endcase
endmodule

`timescale 1ns / 1ps

module first_question_tb;

// Inputs
reg x,y,z;

// Outputs
wire w;

// Instantiate the Unit Under Test (UUT)
```

```
first_question UUT (w,x,y,z);

//Providing input to the UUT
initial begin
// Initialize Inputs
{x,y,z} = 3'b000;

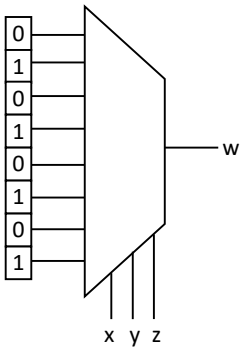
// Wait 10 ns for global reset to finish
#10;

// Add stimulus here
repeat (7)
#10 {x,y,z} = {x,y,z} + 1'b1;
end

//Display the result on the Tcl console (Optional)
initial  begin
$display("   x y z w");
$monitor("\t%b \t%b \t%b \t%b",x,y,z,w);
end

endmodule
```

Multiplexer implementation



Q2) (20 points) You are working with floating-point numbers in your digital system and asked to design a format converter from “half” to “single” form. Implement a Verilog module for the conversion operation. Your module gets input X1 in SEF form and gives output X2 again in SEF form.

SOLUTION:

```
module format_converter(X2,X1);

input [15:0] X1;
output[31:0] X2;

//Sign (S) bits
assign X2[31]= X1[15];

//Exponent (E) bits
assign X2[30:23]= {3'b000, X1[14:10]-5'b01111}+8'h7F;

//Fractional (F) parts
assign X2[22:0]= {X1[9:0], 13'b0000000000000};

endmodule


`timescale 1ns / 1ps

module format_converter_tb;

// Input
reg [15:0] X1;

// Output
wire [31:0] X2;

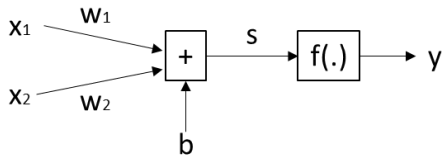
// Instantiate the Unit Under Test (UUT)
format_converter UUT (X2,X1);


//Providing input to the UUT
initial begin
// Initialize Inputs
X1 = 16'hFF00;

end

endmodule
```

Q3) (60 points) Neural networks find extensive usage as a learning system. They are composed of simple elements called “neurons” with a simplified structure as follows.

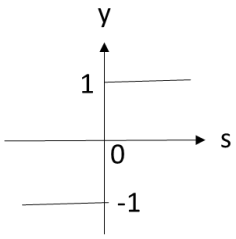


$y=f(x_1*w_1+ x_2*w_2+b)$

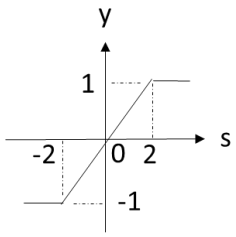
Assume, $0<x_i<0.999$ or $999*10^{-3}$; $0<w_i<0.999$ or $999*10^{-3}$; $0<b<0.999$ or $999*10^{-3}$

f(.) is a transfer function which can be a hard limiter, linear function, or nonlinear function. Here, we will only consider the hard limiter and linear function with below characteristics.

hard limiter function



linear function



Hints:

i- $0<x_i<0.999$ or $999*10^{-3}$ tells us that we can focus on 999 by keeping 10^{-3} in mind only.

ii- $999<1023$ which can be represented as 2^{10}

iii- $0.999*0.999=0.998$

iv- All considered transfer functions have symmetric shape around 0.

a-) **(15 points)** Implement a Neuron module in Verilog with the below template. The user should be able to select which transfer function to use by a control bit.

module Neuron(y,x1,x2,w1,w2,b,tf);
input [9:0] x1,x2,w1,w2,b; //UQ.10 format
input tf; //control bit, tf==0 hard limiter; tf==1 linear fn.
output reg[9:0] y; //UQ.10 format

Form a modular structure with submodules

b-) **(15 points)**
module summer(s,x1,x2,w1,w2,b);
input [9:0] x1,x2,w1,w2,b; //UQ.10 format
output [10:0] s; //UQ1.10 format

c-) **(15 points)**
module hard_limiter(y1,s);
input [10:0] s; //UQ1.10 format
output [9:0] y1; //UQ.10 format

d-) **(15 points)**
module linear_fn(y2,s);
input [10:0] s; //UQ1.10 format
output reg [9:0] y2; //UQ.10 format

SOLUTION:

module summer(s,x1,x2,w1,w2,b);
input [9:0] x1,x2,w1,w2,b; //UQ.10 format
output [10:0] s; //UQ1.10 format

```
wire [19:0] m1,m2;

assign m1=x1*w1;
assign m2=x2*w2;

assign s=m1[19:10]+m2[19:10]+{1'b0,b};

endmodule


module hard_limiter(y1,s);
input [10:0] s; //UQ1.10 format
output [9:0] y1; //UQ.10 format

//assign y1={s[10],10'd999};
assign y1=10'd999;

endmodule

module linear_fn(y2,s);
input [10:0] s; //UQ1.10 format
output reg [9:0] y2; //UQ.10 format

always @(s)
if (s<11'd1998)
y2= s[10:0]/2;
else
y2= 10'd999;

endmodule

module Neuron(y,x1,x2,w1,w2,b,tf);
input [9:0] x1,x2,w1,w2,b; //UQ.10 format
input tf; //tf==0 hard limiter; tf==1 linear fn.

//output [9:0] y; //UQ.10 format

output reg[9:0] y; //UQ.10 format

wire [10:0] s;

summer S1(s,x1,x2,w1,w2,b);

wire [9:0] y1, y2; //UQ.10 format

hard_limiter HL(y1,s);
linear_fn LF(y2,s);

//wire [9:0] mask;
//assign mask={10{tf}};
//assign y=y1&~mask|y1&mask;
```

```
always @(*)
if (tf==1'b0) y=y1;
else y=y2;

endmodule
```